

MPCEMS YATA 改良・改変の全履歴

初期試作から車両同定・大会全体 MPC・ライブ運用・独立受入まで

2026-07-18

目次

1. この資料の読み方	1
2. 一文で表す全体の変化	2
3. 変更の時系列	2
Phase 0: 初期試作版	2
Phase 1: ROS 2 パッケージとして起動可能に再構成	2
Phase 2: PowerShell 一括入口と 3 つの主要モード	2
Phase 3: ライブ運用を自動化	3
Phase 4: WiFi 文字列通信と風予測補正	3
Phase 5: 一画面 dashboard と Grafana	3
Phase 6: 汎用の実測データ入力・車両同定パイプライン	4
Phase 7: 根拠のある物理 map へ再設計	4
Phase 8: BWSC 2025 実ログによる再同定の世代更新	4
Phase 9: 天候データの時刻・地点・独立性を修正	5
Phase 10: 大会全行程・公式規則をモデルへ統合	5
Phase 11: 階層 MPC と実運用向け平滑化	5
Phase 12: 自己学習と GPU 多段方策探索	6
Phase 13: 厳密 1 Hz 受入と mesh convergence	6
Phase 14: 出力 CSV・追跡可能性・安全 gate	6
Phase 15: 教材・運用資料・配布パッケージ	7
Phase 16: 磁気カプラ研究を別系統として高度化	7
4. 現在、結局何をしているのか	7
2026-07-18 18:52 JST の状態	8
5. 完了済みと未完了の境界	8
完了済み	8
未完了	8
6. 現在の品質をどう解釈するか	8
7. GitHub へ掲載するときの短い要約例	9
8. 重要な成果物	9
9. 最後に	9

更新基準日: 2026-07-18

対象: solar_ws0129-main のうち、ソーラーカー EMS を中心とする実質的な改良

用途: GitHub リリースノート、後任者への引継ぎ、設計変更理由の保存

1. この資料の読み方

この資料は、作業中の確認、同じ試験の繰り返し、進捗監視、入力ミスの修正などを省き、パッケージの機能・精度・安全性・使い方を変えた改良だけを時系列で記録したものです。

各段階を次の 4 点で説明します。

- **以前の問題:** なぜ変更が必要だったか。

- **変更内容:** 何を実装または再設計したか。
- **利用者への効果:** 初めて使う人から見て何が変わったか。
- **残る制限:** 現時点で保証していないこと。

ここでいう「同定」は、実測データに最もよく合う車両パラメータを推定する処理です。「MPC」は、将来の天候・道路・電池状態を予測し、一定時間または距離ごとに計画を解き直す制御です。「CEM」は、多数の候補を試し、良かった候補の分布へ探索範囲を寄せていく最適化です。

2. 一文で表す全体の変化

当初のリポジトリは、ソーラーカー用試作 MPC と PASSO 用機能が混在し、起動入口、時刻、相対パス、YAML 反映、実測同定、ライブ通信、長距離シミュレーションが十分に統合されていませんでした。現在は、**実測データ準備 → 車両同定 → 大会環境投入 → 全行程 MPC シミュレーション → ライブ WiFi 運用 → 記録・表示 → 独立検証 → 配布**を同じ profile 構造でつなぐ ROS 2 パッケージへ発展しています。

3. 変更の時系列

Phase 0: 初期試作版

以前の状態

- ソーラーカー MPC と PASSO 燃費支援が同じパッケージに混在していました。
- `mpc_node.py` と `scripts/solar_sim.py` に似た計算が別々にあり、片側だけを修正すると結果がずれる危険がありました。
- `launch` から渡す相対パスが実行ディレクトリに依存し、データを読めない場合があります。
- 2025 年の絶対時刻スケジュールと、起動時刻を基準にする相対予報が混在していました。
- ROS 2 の resource 登録や配布ファイルが不足し、`colcon build` 後の起動が不安定でした。

この段階の位置付け

物理モデルと MPC の骨格はありましたが、「大会で第三者が再現可能に運用する製品」ではなく、研究用コードの集合に近い状態でした。

Phase 1: ROS 2 パッケージとして起動可能に再構成

変更内容

- `resource/mpc_solarcar`、`setup.cfg`、`setup.py` の配布定義を整えました。
- パス解決を共通化し、`source tree`、`install tree`、WSL のどこから起動しても `profile`・`map`・`data` を解決できる構造へ変更しました。
- `solar_state_node.py` を追加し、シミュレーション時に車速、距離、SoC、電圧、電流などの車両状態を自己完結して生成できるようにしました。
- ROS ノードが生存している状態で `rqt graph` を出力する専用処理を追加しました。`graph` 取得専用ノードだけが写る誤った画像を避け、実際の MPC・GPS・logger・dashboard の接続を保存できるようにしました。

利用者への効果

ROS 2 の内部構造を知らない利用者でも、同じ入口からビルド・起動・停止・状態確認・`graph` 保存を行える土台ができました。

Phase 2: PowerShell 一括入口と 3 つの主要モード

変更内容

- Windows 側に `SolarSim.ps1`、WSL/Ubuntu 側に `scripts/solar_control.sh` を設けました。
- `sim`、`measure`、`live` を別 `launch` へ分離しました。
- さらに、オフライン大会全体計算を `simulate`、車両同定を `fit/identify`、気象取得を `forecast` として同じ入口へ統合しました。
- 全入力を `profile` YAML から参照する構造へ変更し、コード中の大会距離、開始時刻、保存先、車両係数の固定値を削減しました。

- YAML 値がノード初期化後に上書きされて効かない問題を修正し、dt、horizon、drive mode、SoC 上下限などを初期化前に確定するようにしました。
- シミュレーション結果を上書きせず、日時・run 名付きの別ディレクトリへ保存し、最新版を指す manifest JSON から追跡できるようにしました。

利用者への効果

利用者は主に profile YAML と入力 CSV を編集し、PowerShell コマンドを選ぶだけで、実測、同定、事前計算、本番運用を切り替えられます。過去結果も自動的に残ります。

Phase 3: ライブ運用を自動化

変更内容

- weather_fetch_node.py を追加し、GNSS 位置に応じて Open-Meteo から予報を定期取得するようにしました。
- solar_autocal_node.py を追加し、発電倍率、走行電力倍率、補機電力を範囲制約付きで更新できるようにしました。
- speed_command_bridge_node.py を追加し、MPC の速度指令に起動待ち、timeout、加減速率制限、deadband、量子化、上制限を適用してから車両へ渡すようにしました。
- distance_node.py と grade_node.py をソーラー運用へ統合し、伴走車 GNSS・高度・車速から距離と勾配を更新できるようにしました。
- solar_preflight_node.py を追加し、テレメトリ、planner、時刻、予報の鮮度を起動前と走行中に判定するようにしました。
- solar_logger_node.py を追加し、車両、伴走車、上位計画、下位指令、天候、校正值、通信状態、安全状態を同じ時系列で保存するようにしました。

利用者への効果

大会開始前から起動しておけば、データ取得、再計画、1 Hz 速度指令、保存、表示が連続して動く構成になりました。センサが途絶えた場合は古い値を無期限に使わず、安全側へ移行します。

Phase 4: WiFi 文字列通信と風予測補正

変更内容

- telemetry_protocol.py で、UDP 文字列の形式、時刻、送信元、フィールド名、単位を定義しました。
- telemetry_text_bridge_node.py で、ソーラーカーと伴走車から届く文字列を検査し、外れ値除去、低域フィルタ、変化率制限を通して ROS topic へ変換するようにしました。
- 車速、距離、電池、風、GNSS などについて、上限、逆行許容量、timeout を個別設定できるようにしました。
- 車両側・伴走車側・planner 側の送受信サンプルを追加し、Raspberry Pi やマイコン側の実装見本を用意しました。
- wind_correction_node.py を追加し、実測風と予報風の差を距離・時間相関で将来へ伝搬し、補正予報と信頼区間を生成するようにしました。

利用者への効果

同一 WiFi 内の Raspberry Pi は、決められた 1 行文字列を UDP で送るだけで ROS 2 へ接続できます。ROS 2 を車両側へ入れなくても、伴走車 PC が全情報を統合できます。

Phase 5: 一画面 dashboard と Grafana

変更内容

- 初期 dashboard を、数値カード中心の一画面表示へ圧縮しました。
- 車速指令、実車速、SoC、距離、発電、消費、電圧、電流、風、勾配、温度、通信鮮度、安全状態をスクロールなしで確認できる配置へ整理しました。
- dashboard_node.py へ JSON API に加えて Prometheus 形式の /metrics を追加しました。
- grafana/へ Prometheus と Grafana の provisioning を追加し、SolarSim.ps1 -Action grafana で同じ画面を再現できるようにしました。

利用者への効果

ブラウザを開くだけで運転判断に必要な値を一画面で確認でき、表示設定を各 PC で手作業再構築する必要がなくなりました。

Phase 6: 汎用の実測データ入力・車両同定パイプライン

変更内容

- raw CSV 雛形と sample データを追加し、必要な列、単位、時刻形式、欠損時の扱いを定義しました。
- GPS から route profile を作る処理、OCV-SoC 曲線、Rint 温度/SoC マップ、PV/MPPT マップ、駆動/回生マップを作る処理を追加しました。
- run_identification_pipeline.py を簡易入口、run_vehicle_identification.py をセグメント分割・multi-start・joint refinement を含む本格入口として整備しました。
- 係数だけでなく、map の形状補正、温度依存、SoC 依存、ライン抵抗、分極抵抗・時定数、grade scale、headwind exposure も推定対象へ拡張しました。
- 同定結果から候補 profile、map、fit summary、replay CSV、図、TeX/PDF を自動生成し、手作業の転記をなくしました。
- 同定候補は自動で本番 profile へ昇格させず、独立 validation gate を通った場合だけ promote できる構造へ変更しました。

利用者への効果

別車両でも、決められた CSV へ実測ログを入れて同じコマンドを実行すれば、その車両用の model/maps/profile 一式を作成できます。どの値が実測、仕様書、理論、推定かも追跡できます。

Phase 7: 根拠のある物理 map へ再設計

以前の問題

初期 map には、製品仕様書や実測値との対応が弱い仮置き形状があり、map 全体へ倍率を掛けるだけでは実車再現性を説明できませんでした。

変更内容

- モーター/コントローラは回転数、トルク、銅損、鉄損、インバータ損失を基礎に eco/power map を分離しました。
- 回生は駆動 map の単純コピーをやめ、回生可能電流、低速域、充電効率を別に扱いました。
- 電池は OCV-SoC、Rint 温度/SoC、配線抵抗、分極抵抗と時定数を Thevenin 形で分離しました。
- パネルは面積、基準効率、温度係数、入射日射、panel map、MPPT map を分離しました。
- 仕様書・試験結果・チーム観測・推定値を evidence bundle に保存し、根拠のない自由度を増やさない方針へ変更しました。

利用者への効果

同じ RMSE でも「たまたま係数が相殺したモデル」と「部品の物理に沿ったモデル」を区別できます。将来部品を交換した場合も、該当 map だけを差し替えやすくなりました。

Phase 8: BWSC 2025 実ログによる再同定の世代更新

変更内容

- day1 から day6 の大会ログだけを本戦ログとして扱い、サーキット試走などを分離しました。
- 公式 control stop、短いトラブル停車、日跨ぎ、最終事故を別イベントとして扱いました。
- 車重を 235 kg、走行中補機を約 21 W、夜間補機を 0 W、新品電池を 3011 Wh、CdA を設計値 0.093 より悪化した約 0.11 近傍という物理事前条件で再同定しました。
- 70 km/h 巡航で約 800 W というチーム分析を独立の sanity check として使いました。
- MLE 世代を進める中で、瞬時気象、回生同期、距離窓エネルギー、慣性電力、区間別 grade、停止中充電、終端 SoC アンカーを順に追加しました。
- 現在のパッケージ世代は bwsc2025_fitted_mle19_energywindow_inertia、その内部の最新研究同定 run は mle35_expanded_grade_single_source_ultra_v1 です。番号は「パッケージ世代」と「内部実験 run」で役割が異なります。

現在の研究候補値

- mass: 235 kg
- CdA: 約 0.1112 m²
- Crr: 約 0.00626
- driving auxiliary: 約 21.02 W
- nominal energy: 3011 Wh
- panel gain: 約 0.7127
- conditional clean power RMSE: 約 201.34 W
- end-to-end clean power RMSE: 約 246.09 W
- battery-conditioned voltage RMSE: 約 0.968 V

残る制限

70 km/h 巡航中央値は実測約 800.05 W に対してモデル約 818.05 W で近い一方、加減速、低速、Day 1/4/6、終端電圧に残差があります。終端 SoC の独立根拠は約 19.64 percentage points の幅を持つため、MLE35 は研究候補であり、本番 canonical profile へ自動昇格していません。

Phase 9: 天候データの時刻・地点・独立性を修正

変更内容

- 予報 CSV の値が「その時刻の瞬時値」か「前 1 時間平均」かを metadata で明示しました。
- 走行地点と時刻に対応する Open-Meteo archive の GHI/DNI/DHI、気温、風を route-time grid へ変換しました。
- GHI からパネル面への POA へ変換する経路を追加しました。
- 実測 PV から逆算した effective irradiance は同定診断専用とし、将来方策の性能評価には使用禁止としました。
- 通常の日射列へ物理上限を設け、補正前の有効値は別列として保存しました。
- check_policy_weather_input.py を追加し、ファイル名だけでなく全行の source、product role、時刻意味を検査し、PV 由来の循環天候を GPU 探索・厳密受入へ入れられないようにしました。

利用者への効果

「車両の発電実績で補正した晴天」を同じ車両の最適化に再利用して、性能を楽観的に見せる循環評価を防げます。Day 5/6 の 600 から 700 W/m² は日平均ではなく瞬時上位値であり、日平均はそれぞれ約 433、417 W/m² として区別されます。

Phase 10: 大会全行程・公式規則をモデルへ統合

変更内容

- リタイア地点 2831 km で計算を打ち切る方式をやめ、BWSC 2025 の全 3026.9 km を最適化対象にしました。
- 公式 program に基づく 9 か所の control stop、各 30 分、opening/closing 時刻を入力しました。
- Adelaide の最終締切を 2025-08-29 17:30 local、すなわち 08:00 UTC として絶対制約にしました。
- 早着時は opening まで待機、closing 後到着または finish deadline 超過は非可行としました。
- no-trouble 実験では公式停車だけを残し、実走行時の故障停車と 2831 km の車体破損を除外しました。
- 停車中・夜間は走行用補機を 0 W とし、朝夕の静止充電とパネル傾斜条件を別に扱いました。

利用者への効果

「実ログを再現するシミュレーション」と「故障がなかった場合の反実仮想 MPC」を混同せず比較できます。完走判定は距離だけでなく、全 control stop と最終締切を含みます。

Phase 11: 階層 MPC と実運用向け平滑化

変更内容

- 上位 planner を大会残距離の distance-domain MPC として整理し、速度、時間、SoC、電池温度を予測するようにしました。
- 下位 planner を 1 Hz で動かし、上位速度参照への追従と指令変化を同時に最小化するようにしました。
- 指令には加減速率、slew、deadband、measurement timeout、low-pass filter を追加しました。

- 上位目的関数を待ち時間、走行時間、終端/日末 SoC、速度平滑性、電流二乗、Joule 損、空力/機械エネルギー、電力 slew、温度、制約 barrier へ分解しました。
- `upper_policy.py` を追加し、offline 学習済み方策は live MPC を固定するのではなく、初回の warm start として利用するようにしました。以後は最新の計測と予報で receding-horizon 再計画します。

利用者への効果

ノイズで速度指令が上下する問題を抑えつつ、予報や SoC が変わった場合は計画を更新できます。認証用の固定方策と本番用の再計画 MPC を別 profile に分離しました。

Phase 12: 自己学習と GPU 多段方策探索

変更内容

- `tune_upper_planner_weights.py` で、人間の速度計画を正解として模倣せず、完走時間、制約、終端エネルギーから目的関数重みを CEM で探索できるようにしました。
- TensorBoard へ世代、候補分布、best score、制約を保存するようにしました。
- `gpu_upper_policy_search.py` で、距離ごとの速度方策を CUDA 上で一括評価できるようにしました。
- 全 3026.9 km を、粗探索 5 km 積分/25 km 制御、fine 1 km/5 km、ultra 0.1 km/5 km、さらに 2 km と 1 km 制御へ段階的に細分化しました。
- 制御変数の次元は 123、607、1515、3028 へ増えます。物理積分は最終的に約 30,269 個の 100 m 遷移を持ちます。
- 4 独立 seed、8400 世代、合計 32,993,280 候補を Slurm dependency chain で処理し、checkpoint と profile SHA を各段に固定しました。
- 100 m 段の過大 population を実測所要時間に基づいて 256 へ修正し、24 時間 job limit 内で完走可能な構成にしました。

利用者への効果

旧 4 点速度計画では表現できなかった地点別速度差を、道路・天候・control stop に合わせて探索できます。GPU 探索は MPC の代替ではなく、live MPC の高品質な初期解を作る役割です。

Phase 13: 厳密 1 Hz 受入と mesh convergence

変更内容

- GPU surrogate の結果をそのまま採用せず、`solar_sim.py` で固定方策を 100 m 予測・1 Hz 実行として再生する受入段を追加しました。
- 固定方策評価が同じ 30,269 区間を 2 回計算していた冗長処理を 1 回へ削減しました。
- 5 km、2 km、1 km 制御方策を別々に最適化し、同じ方策を補間しただけの偽の mesh 比較を禁止しました。
- 3 格子 x 4 seed を最大 12 CPU で並列評価し、各 seed の結果を独立保存した後、最速の可行候補だけを merge するようにしました。
- 時間差、終端 SoC 差、速度 RMS、prediction/execution SoC 差に受入閾値を設けました。
- 数値方策合格を POLICY_ACCEPTANCE_COMPLETE、車両 model も含む本番昇格を ACCEPTANCE_COMPLETE として分離しました。

利用者への効果

CUDA 上の近似計算が速いだけでは本番採用されません。離散化を細かくしても結果が変わらず、1 秒刻みで電圧・電流・SoC・時刻制約を守ることを確認して初めて方策を採用できます。

Phase 14: 出力 CSV・追跡可能性・安全 gate

変更内容

- summary CSV、detail CSV、upper plan、1 Hz lower command、HTML report、manifest JSON を run ごとに保存するようにしました。
- detail CSV へ GHI/DNI/DHI/POA、panel/MPPT 効率、PV 電力、走行/回生/補機電力、OCV、Rint、電圧、電流、SoC、温度、勾配、風、速度制限、計画/実行速度を追加しました。

- `step_dt` が 600 秒から端数へ変化する理由を、区間末端、stop 境界、日末、control stop 境界へ正確に合わせる可変最終 step として明示しました。
- model validation、weather independence、surrogate feasibility、exact replay、mesh convergence を別 gate にしました。
- profile、map、入力 CSV、出力成果物へ SHA-256 と manifest を付け、異なる世代を混ぜないようにしました。

利用者への効果

結果の数字だけでなく、「どの入力と model で、各秒に何 W 発電し、何 W 使い、なぜその SoC になったか」を後から追跡できます。

Phase 15: 教材・運用資料・配布パッケージ

変更内容

- README を、入口、各 mode、入力、保存先、内部処理、model 世代、安全 gate まで含む体系へ拡張しました。
- オールインワン解説書、導入運用書、全フローワークブック、MLE 手計算ワークブックを TeX/PDF で作成しました。
- 問題編、解答用紙、完全解答を分離し、map 補間、車両抵抗、PV、電池 IV、上位 CEM/L-BFGS-B、下位 MPC、UDP、MLE、保存先を手計算できるようにしました。
- PDF は文字重なり、切れ、空白ページ、未解決参照を修正し、主要ページの画像確認用成果物を保存しました。
- `create_solarcar_only_package.py` で PASSO、磁力、build/install/log、旧 MLE 世代を除いたソーラーカー専用配布物を生成できるようにしました。
- `create_solarcar_blank_package.py` で、同じ機能と資料を保ちながら車両・大会データだけを空にした新車両用 template を生成できるようにしました。

利用者への効果

研究者本人以外のチームメンバーが、GitHub ZIP の取得から、Windows/Ubuntu 設定、実測、同定、sim、live、マイコン通信、障害復旧まで学べる形になりました。

Phase 16: 磁気カプラ研究を別系統として高度化

この変更はソーラーカー EMS の本番配布には含めませんが、root repository 内の別研究系統として実施されました。

変更内容

- 単純磁場表示から、円柱磁石、固定高さ自由配列、有限変位復元力、yaw 復元、clearance、package、site spacing を扱う高忠実度モデルへ拡張しました。
- GPU dipole screening、有限モデル再評価、Fourier topology、adaptive random、CEM を段階分離しました。
- 正定値剛性、並進復元、yaw 復元、負復元なし、clearance、package、線形性、直交漏れ、条件数、中心釣合いなど 16 gate を設けました。
- surrogate 合格後に円柱磁石の exact static 評価を行い、gate 通過候補だけを STEP へ出力する構成にしました。
- `reevaluate_fourier_gpu_candidates.py` の `inner_support_points_mm` は、exact 評価で最小内側支持半径を計算するための正式な設計保存値です。

分離理由

磁気カプラは車両 EMS の ROS runtime に不要です。root 研究 repository では保持しますが、solar-only release では削除し、運用者が誤って起動しないようにしています。

4. 現在、結局何をしているのか

2026-07-18 時点で行っているのは、MLE35 研究候補 model を使った、故障なし BWSC 2025 全 3026.9 km 方策の GPU 多段探索です。

処理順は次のとおりです。

1. 25 km ごと 123 変数の粗い速度方策を、5 km 物理積分で 1800 世代探索する。
2. 5 km ごと 607 変数へ増やし、1 km 物理積分で 200 世代改善する。
3. 同じ 607 変数を 100 m 物理積分で 60 世代再調整する。

4. 制御間隔を 2 km、次に 1 km へ細かくし、それぞれ 20 世代調整する。
5. 4 つの独立 seed が全部終わったら、最良候補を統合する。
6. GPU 近似とは別の `solar_sim.py` で、100 m 予測と 1 Hz 実行の full simulation を行う。
7. 5/2/1 km 方策の mesh convergence、公式時刻、電流、電圧、SoC を判定する。
8. 方策 gate と車両 model gate の両方を通った場合だけ live 用 profile へ昇格する。

2026-07-18 18:52 JST の状態

- seed 0: coarse、fine、100 m、2 km control、1 km control を完了。
- seed 1: coarse と fine を完了し、100 m 段を実行中。
- seed 2、seed 3: GPU 割当待ち。
- campaign finalizer: 4 seed 完了待ち。
- exact 1 Hz acceptance: campaign finalizer 待ち。
- エラー: なし。GPU 使用枠が 1 枚のため直列進行。

seed 0 の暫定値は、1 km control 方策で約 125.4452 時間、終端 SoC 約 10.549%、公式時刻違反 0 秒です。ただし、これは GPU surrogate 値であり、厳密 1 Hz 合格値ではありません。

5. 完了済みと未完了の境界

完了済み

- ROS 2 の sim、measure、live、live WiFi 入口。
- profile YAML による入力・出力管理。
- WiFi 文字列 protocol、filter、timeout、安全指令 bridge。
- Open-Meteo 取得、風補正、online bounded calibration。
- logger、preflight、one-screen dashboard、Grafana。
- 汎用 CSV 同定、map 生成、候補 profile、報告書生成。
- BWSC 2025 全 3026.9 km、9 control stops、finish deadline のモデル化。
- MLE35 研究候補と独立 model gate。
- GPU 多段探索、checkpoint、weather gate、exact acceptance pipeline の実装。
- solar-only release と blank release の生成器。
- README、運用書、ワークブック群。

未完了

- 現在の 4-seed GPU campaign の全完走。
- その結果に対する 100 m/1 Hz 厳密 full simulation。
- mesh convergence 後の最終方策 PDF。
- MLE35 の本番 canonical profile への昇格。
- 終端 SoC 証拠幅 19.64 points を縮める新しい独立実測データの取得。

6. 現在の品質をどう解釈するか

現在の planner 探索が長時間かかっている理由は、単に「数千世代だから」だけではありません。1 候補の中で、全 3026.9 km、日跨ぎ、9 停車、天候、map、電池 IV、温度を順番に計算するためです。さらに 4 seed と複数 mesh を使い、偶然良かった 1 回を採用しない設計にしています。

一方、世代数を増やしても実測データの不足は解消しません。planner の最適化誤差と vehicle model の同定誤差を分離するため、次の 3 段階を守ります。

1. GPU で良い方策候補を探す。
2. exact 1 Hz replay で数値方策を検証する。
3. 独立ログで vehicle model を検証する。

第 2 段まで通っても第 3 段が落ちた場合、その方策は研究用であり、本番採用しません。

7. GitHub へ掲載するときの短い要約例

YATA ソーラーカー EMS を、試作 ROS 2 ノード群から、車両同定・大会全体 MPC・ライブ WiFi 運用・記録・Grafana 表示・独立検証・配布まで一貫した profile 駆動パッケージへ再構成しました。BWSC 2025 の全 3026.9 km、公式 9 control stops、最終締切、夜間補機 0 W を実装し、実ログと独立気象から物理 map を再同定しました。上位方策は 4 seed、8400 世代、約 3300 万候補の multi-fidelity CUDA 探索を行い、100 m 予測・1 Hz full replay・mesh convergence・vehicle-model gate を通った結果だけを live MPC の warm start へ昇格します。PASSO/磁力/旧成果物を除いた solar-only release と、データ空の base template、初心者向け運用書・手計算教材も追加しました。

8. 重要な成果物

- 主要 README: README.md
- PowerShell 入口: SolarSim.ps1
- ROS ノード: mpc_solarcar/
- simulation launch: launch/solarcar_sim.launch.py
- measurement launch: launch/solar_measurement.launch.py
- live launch: launch/solar_race_live.launch.py
- live WiFi launch: launch/solar_race_live_wifi.launch.py
- 本格同定: scripts/run_vehicle_identification.py
- full simulation: scripts/solar_sim.py
- GPU 探索: scripts/gpu_upper_policy_search.py
- multi-fidelity 投入: scripts/submit_solar_gpu_multifidelity_campaign.sh
- exact 受入: scripts/run_solar_gpu_acceptance_pipeline.sbatch
- Grafana: grafana/
- 現行研究 package: project_packages/下の bwsc2025_fitted_mle19_energywindow_inertia/
- 新車両 template: project_packages/bwsc2027_template/
- 統合資料: docs/solar_all_in_one_manual/
- 全フロー手計算教材: docs/complete_flow_workbook/
- MLE 手計算教材: docs/mle_hand_calculation_workbook/
- 同定済み版の配布生成: scripts/create_solarcar_only_package.py
- 空版の配布生成: scripts/create_solarcar_blank_package.py

9. 最後に

最大の改良は、単に MPC の係数を変えたことではありません。入力データの根拠、物理 model、最適化、1 Hz 実行、独立 validation、保存先、資料、配布物を一つの追跡可能な流れへつないだことです。

現在の GPU 結果が良好でも、それだけで「YATA が必ず完走する」「連続問題の大域最適解が証明された」「vehicle model が高精度」とは扱いません。定義した有限候補集合での探索、mesh convergence、exact replay、独立 model gate を順に通し、合格範囲を明示して初めて運用へ移します。